Online Training Programs
ASP.NET Core Web API Online Training

ASP.NET Web API – Basics
Introduction to ASP.NET Web API
Creating ASP.NET Web API Application
How to add Swagger in Web API
How to use Fiddler to test Web API
How to use POSTMAN to test Web API
ASP.NET Web API using SQL Server
Content Negotiation in Web API
Media Type Formatter in Web API
How to Implement GET Method in Web API
How to Implement POST Method in Web API
How to Implement PUT Method in Web API
How to Implement DELETE Method in Web API
Custom Method Names in Web API
Parameter Binding in Web API
Consuming Web API Service From jQuery
Calling Web API Service in a Cross Domain Using jQuery AJAX
Cross-Origin Resource Sharing in WEB API

WEB API Routing and Attribute Routing
ASP.NET Web API Routing
Routing Variations in WEB API
Routing and Action Selection in Web API
ASP.NET Web API Attribute Routing
Optional Parameters in Web API Attribute Routing
Route Prefix in Web API
Web API Attribute Routing
Route Constraints
Route Names and Route Orders in Attribute Routing

ASP.NET Web API – Security
Authentication and Authorization in ASP.NET Web API
ASP.NET Web API Basic Authentication
Role Based Basic Authentication in Web API
Web API Service with Basic Authentication
HTTP Message Handlers in WEB API
Basic Authentication Using Message Handler in Web API
HTTP Client Message Handler in Web API
Token Based Authentication in ASP.NET Web API
Client Validation Using Basic Authentication in Web API
Refresh Token in Web API
Consume Refresh Token in C# Client
HMAC Authentication in Web API

Web API Versioning
Web API Versioning Using URI
Web API Versioning using Query string Parameter
Web API Versioning Using Custom Header
Web API Versioning Using Accept Header
Web API Versioning Using Custom Media Types

Advanced Concepts
Enable SSL in Visual Studio
Consume OAuth Token
Enable HTTPS in Web API

Popular ASP.NET Web API Books
Most Recommended ASP.NET Web API Books

Basic Authentication Using Message Handler in Web API

Back to ASP.NET Web API Tutorials for Beginners and Professionals

Basic Authentication Using Message Handler in Web API

In this article, I am going to discuss how to implement **Basic Authentication Using Message Handler** in ASP.NET Web API. Please read our last article, where I discussed the **Server-Side HTTP Message Handler** in ASP.NET Web API.

As we already discussed, the basic authentication says that the client needs to send the username and password in base64 encoded format in the authorization header of the HTTP request. The server then gets the username and password from the authorization header. Once the username and password are extracted from the header, then the server does and match the credentials with any persistent storage (most of the time it may be a database). If the credentials are found in the persistent storage then the server will treat that HTTP request as a valid request and process it else it simple return unauthorized response to the client.

Let's us Implement Basic Authentication Using Message Handler

We are going to use the following Database table in this demo.

UserID	UserName	UserPassword	UserRoles	UserEmail
101	Anurag	123456	Admin	Anurag@g.com
102	Priyanka	654321	User	Priyanka@g.com
103	Pankaj	123456	Admin,User	Pankaj@g.com
104	Raghav	abcd123	Admin, User	Raghav@g.com

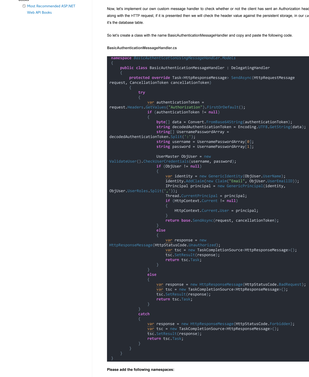
Please use below SQL Script to create and populate the UserMaster table with the required sample data.

```
--Create a new database called 'DemoDB'
--Connect to the master database
CREATE DATABASE DemoDB
GO

USE [DemoDB].[dbo]
GO
--Create a table to store the user information
CREATE TABLE [dbo].[UserMaster]
(
    [UserID] INT PRIMARY KEY,
    [UserName] VARCHAR(50),
    [UserPassword] VARCHAR(50),
    [UserRoles] VARCHAR(100),
    [UserEmail] VARCHAR(100)
)
GO

INSERT INTO UserMaster VALUES(101, 'Anurag', '123456', 'Admin', 'Anurag@g.com')
INSERT INTO UserMaster VALUES(102, 'Priyanka', '654321', 'User', 'Priyanka@g.com')
INSERT INTO UserMaster VALUES(103, 'Pankaj', '123456', 'Admin,User', 'Pankaj@g.com')
INSERT INTO UserMaster VALUES(104, 'Raghav', 'abcd123', 'Admin, User', 'Raghav@g.com')
GO
```

Let's create an empty Web API application with the name BasicAuthenticationUsingMessageHandler (you can give any name) and select empty and Web API as shown in the below image.



Once you click on the OK button, it will create the application for us. Then the next step is to create an ASP.NET Entity Data Model against the SecurityDB and Select the UserMaster table.

Here you need to choose DB First Approach of Entity Framework.

Let's create a class with the name ValidateUser and copy and paste the following code.

```
namespace BasicAuthenticationUsingMessageHandler.Models
{
    public class ValidateUser
    {
        //this method is used to check the user credentials
        public static UserMaster CheckUserCredentials(string username, string password)
        {
            // Console.WriteLine("Inside CheckUserCredentials");
            using (var context = new DemoDBEntities())
            {
                return context.UserMasters.FirstOrDefault(user =>
                    user.UserName.Length == username.Length && StringComparison.OrdinalIgnoreCase)
                    && user.UserPassword == password);
            }
        }
    }
}
```

In the above class, we create one method i.e. CheckUserCredentials which will validate the user by checking the username and password.

Now, let's implement our own custom message handler to check whether or not the client has sent an Authorization header along with the HTTP request. If it is presented then we will check the header value against the persistent storage, in our case, it's the database table.

So let's create a class with the name BasicAuthenticationMessageHandler and copy and paste the following code.

BasicAuthenticationMessageHandler.cs

```
namespace BasicAuthenticationUsingMessageHandler.Handlers
{
    public class BasicAuthenticationMessageHandler : DelegatingHandler
    {
        protected override Task<HttpResponseMessage> SendAsync(HttpRequestMessage
request, CancellationToken cancellationToken)
        {
            try
            {
                var authenticationToken =
request.Headers.GetValues("Authorization").FirstOrDefault();
                if (!authenticationToken != null)
                {
                    byte[] data = Convert.FromBaseStringToArray(authenticationToken);
                    string decodedAuthenticationToken = Encoding.UTF8.GetString(data);
                    string[] UserNamePasswordArray = //
decodedAuthenticationToken.Split(':');
                    string username = UserNamePasswordArray[0];
                    string password = UserNamePasswordArray[1];
                    //
                    UserMaster CheckUser = //
ValidateUser.CheckUserCredentials(username, password);
                    if (CheckUser != null)
                    {
                        var IdentityArg = new GenericIdentity(CheckUser.UserName);
                        IdentityArg.AddClaim(new Claim("Email", CheckUser.UserEmail));
                        IPrincipal principal = new GenericPrincipal(IdentityArg,
CheckUser.UserRoles.Split());
                        Thread.CurrentPrincipal = principal;
                        if (HttpContext.Current != null)
                        {
                            HttpContext.Current.User = principal;
                        }
                        return base.SendAsync(request, cancellationToken);
                    }
                    else
                    {
                        return base.SendAsync(request, cancellationToken);
                    }
                }
                else
                {
                    var response = new HttpResponseMessage(HttpStatusCode.BadRequest);
                    var tsc = new TaskCompletionSource<HttpResponseMessage>();
                    tsc.SetResult(response);
                    return tsc.Task;
                }
            }
            catch
            {
                var response = new HttpResponseMessage(HttpStatusCode.Forbidden);
                var tsc = new TaskCompletionSource<HttpResponseMessage>();
                tsc.SetResult(response);
                return tsc.Task;
            }
        }
    }
}
```

That's it. We are done with our implementation. Let's create one empty Web API controller and then copy and paste the following code.

```
using System.Net;
using System.Net.Http;
using System.Threading;
using System.Web.Http;

namespace BasicAuthenticationUsingMessageHandler.Controllers
{
    public class TestController : ApiController
    {
        [AuthorizeRoles = "Admin,User"]
        public HttpResponseMessage Get()
        {
            //You can implement your own logic
            //use the identity name
            string username = Thread.CurrentPrincipal.Identity.Name;
            return Request.CreateResponse(HttpStatusCode.Ok, "User Name - " + username);
        }
        [AuthorizeRoles = "Admin"]
        public HttpResponseMessage Post()
        {
            string username = Thread.CurrentPrincipal.Identity.Name;
            return Request.CreateResponse(HttpStatusCode.Ok,"User Name - " + username);
        }
    }
}
```

As you can see in the above controller, both the Get and Post Action are decorated with an Authorize attribute and we have specified the role over each action. So, the specific roles can access a specific action. Since the Get() is to read the data, generally both the Admin and the User can access it but a Post is only allowed for an Admin.

Testing using Postman

First, let's test for the following user

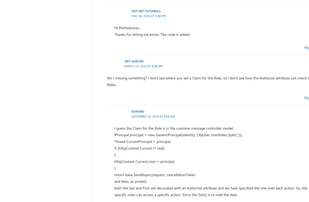
UserName: Priyanka

Password: abcdef

The username and password need to be a colon (:) separated and must be in base64 encoded. To do this use the following website

https://www.base64encode.org/

Enter the username and password separated by a colon (:) in **"Encode to Base64 format"** textbox, and then click on the **"Encode"** button as shown in the below diagram which will generate the Base64 encoded value.

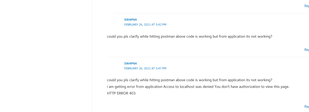


Once you generated the Base64 encoded string, let's see how to use basic authentication in the header to pass the Base64 encoded value.

Here we need to use the Authorization header and the value will be the **Base64 encoded** string as shown below.

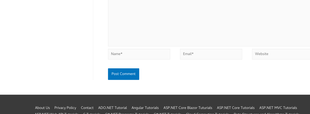
Authorization: UHJpeWFuazphYmNkZWY=

GET Request:



As you can see, we get Status 200 as expected as the user Priyanka has the role 'User' and the role 'User' has access to the Get Method of the Test Controller.

POST Request:



As you can see, we get Status 401 Unauthorized as expected as the user Priyanka has the role 'User' and the role 'User' does not have access to the Post Method of the Test Controller.

In the next article, I am going to discuss **HTTP Client Message Handler** with some examples. Here, in this article, I try to explain the **Basic Authentication Using Message Handler** step by step with an example. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.

Dot Net Tutorials
About the Author: Pranaya Rout
Pranaya Rout has published more than 3,000 articles in his 11 year career. Pranaya Rout has very good knowledge on Microsoft technologies like ASP.NET Core, ASP.NET MVC, ASP.NET Web API, ASP.NET Web Forms (EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

6 thoughts on "Basic Authentication Using Message Handler in Web API"

MUTHUKUMAR
MAY 18, 2015 AT 5:55 PM

BasicAuthenticationMessageHandler code was not there
Can you please provide?

Reply

DOT NET TUTORIALS
MAY 18, 2015 AT 5:48 PM

Hi Muthukumar,
Thanks for letting me know. The code is added.

Reply

JEFF SANTOS
MAY 21, 2015 AT 8:46 PM

Are I missing something? I don't see where you set a Claim for the Role, so I don't see how the Authorize attribute can check the Roles.

Reply

EDWARD
DECEMBER 16, 2015 AT 8:06 AM

I guess the Claim for the Role is in the custome message controller model:
IPrincipal principal = new GenericPrincipal(IdentityArg, CheckUser.UserRoles.Split(';'));
Thread.CurrentPrincipal = principal;
if (HttpContext.Current != null)
{
HttpContext.Current.User = principal;
}
return base.SendAsync(request, cancellationToken);
and then, as posted:
both the Get and Post are decorated with an Authorize attribute and we have specified the role over each action. So, the specific roles can access a specific action. Since the Get() is read the data

Reply

DEEPTHI
FEBRUARY 26, 2021 AT 3:42 PM

could you pls clarify while letting postman above code is working but from application its not working?

Reply

DEEPTHI
FEBRUARY 26, 2021 AT 3:47 PM

could you pls clarify while letting postman above code is working but from application its not working?
I am getting error from application Access to localhost was denied You don't have authorization to view this page.
HTTP 10061K ecb

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name * Email * Website

Post Comment

© Dot Net Tutorials | Website Design by Sunrise Pixel